Lossy Calibration Archives for Exact Model Repair

Anonymous TACL submission

Abstract

Low-bit language models use calibration text to choose a small set of allowed weight values. Removing calibration records can change those choices. Recovering the revised model usually requires the original text or a large archive of its internal features. We show how a lossy feature archive can still recover the exact weight choices. The archive stores intervals, and a verifier accepts a choice only when every feature value in those intervals leads to it. Unresolved choices trigger source replay. For a calibration procedure whose feature extractor stays fixed, we prove exact model recovery and equality of the state needed for later deletions. Complete reconstructions of DistilGPT2 and GPT2 match fresh reconstruction across 42.47 and 84.93 million weight codes. In a matched study, compression saves 7.01–7.86% of live storage while taking 1.44–2.66 times the optimized lossless request time. The result is a checked storage and source-access tradeoff: exact discrete model decisions can survive the loss of exact feature values.

1 Introduction

A language model can be made smaller by replacing each weight with one of a few allowed values. Calibration text helps choose these values by showing which weight errors matter for the model's internal computations. That text is often treated as a one-time input. Yet a calibration collection can later change. A record may be duplicated, corrupted, or removed from a deployment's approved collection. The resulting maintenance task is concrete: rebuild the low-bit weights for the retained records and keep enough state to process the next removal.

The obvious solutions retain different kinds of information. A service can keep the text and extract its features again. It can instead retain all extracted feature values, avoiding the original text during repair. It can also aggregate features into matrices that summarize their effect on calibration. These choices have different storage costs and different source-access requirements. A small aggregate alone does not necessarily contain the contribution of the record that must be removed.

We study a fourth option: retaining compressed intervals around the feature values. At first this seems incompatible with exact reconstruction. If compression loses a feature value, how can a later computation be exact? The key is that the required output is discrete. Many possible feature values lead to the same rounded weight. The archive need only preserve enough information to distinguish the relevant rounding choices. It need not reconstruct every original feature bit.

This distinction is useful even when deletion changes the model. Our verifier recomputes the decisions for the retained set. It does not merely test whether the old model remains unchanged. Each accepted decision is valid for every feature matrix described by the archive. A numerical solver proposes the calculation; directed bounds decide whether that proposal supports a unique weight code. When the stored information is insufficient, the service detects the ambiguity and can regenerate exact features from retained sources.

The reconstruction target must make this comparison meaningful. We fix a feature extractor from the original checkpoint before choosing calibration membership. Each record then has the same features regardless of which other records remain. This *fixed-teacher calibration* supports source-local archives and exact subtraction of source contributions. Ordinary sequential calibration instead allows earlier quantized layers to

alter later features. We measure the predictive cost of the fixed-teacher choice separately. The task changes calibration membership; it does not remove information from the checkpoint's pretraining.

Our contribution has three parts. First, we specify a repair service that uses lossy feature enclosures to certify adaptive weight decisions, including changed codes. Second, we connect executable residual bounds to those decisions and prove that a successful repair produces both the required model and a canonical successor state. Third, we evaluate complete transactions against cold reconstruction, two lossless caches, and an exact-Gram hybrid. The experiments separate storage, source access, request time, and prediction quality.

The practical question is therefore which information a service should retain. A compressed archive offers exact output with fewer stored bytes when its intervals are decisive. A lossless archive spends more storage to simplify the numerical work. A Gram-based service can be smaller still when it may revisit source records. We make these choices comparable by holding the reconstruction target fixed and charging the whole request, rather than only its fastest kernel.

2 What must the archive remember?

Consider a service maintaining a four-bit model. Four-bit weights use sixteen allowed values, called *codes*. Calibration adjusts one weight at a time. An earlier rounding error influences the next choice through coefficients computed from the calibration features. Deletion changes those coefficients, which can change later codes. Exact repair therefore requires agreement along this entire adaptive sequence.

A changed code from uncertain features. A two-weight example illustrates the information requirement. Let the original weights be $(0.4, 0.4)$, with allowed values $\{-1, 0, 1\}$. Suppose two records have feature vectors $(1, 1)$ and $(-1, 1)$. Their cross-products cancel. The first weight rounds to zero, and the second receives no correction, so the output is $(0, 0)$. Now delete the second record. With a ridge term of one, the coefficient carrying the first rounding error into the second coordinate is

$$c = \frac{x_1 x_2}{1 + x_2^2} = \frac{1}{2}.$$

The second adjusted value becomes $0.4 + c(0.4 - 0) = 0.6$, and its code becomes one.

Suppose the archive retains only $x_1, x_2 \in [0.99, 1.01]$. Direct interval bounds give

$$\frac{0.9801}{2.0201} \le c \le \frac{1.0201}{1.9801}.$$

Every resulting adjusted value lies inside $[0.59, 0.61]$, entirely above the rounding boundary 0.5. Thus the revised output $(0, 1)$ is exact even though the archive cannot identify either original feature value. This is an algebraic illustration, not an experimental result. If the interval crossed 0.5, the same check would request more information rather than guess a code.

Three separate objects. It helps to distinguish the model, its repair state, and its execution history. The model contains the deployed codes. The repair state identifies retained records and stores the evidence needed for a later deletion. The history records which computation was attempted, including source replay and failure. Two methods can produce identical model codes while keeping different repair states. Two executions can produce identical canonical states while having different costs. We compare each object under its own contract.

Where compression can help. A source-local archive removes a deleted record by dropping its descriptor. For the surviving records, certification may avoid both exact feature storage and source access. This is attractive when retaining activations is expensive and revisiting the corpus is inconvenient. It is also a stricter requirement than obtaining similar predictions: the output must equal a specified reconstruction, including decisions near rounding boundaries. The benefit depends on decision margins as well as compressibility. Narrow intervals help only when they resolve the particular adaptive choices made after deletion.

The alternative access contracts. An exact feature cache can reconstruct a retained Gram matrix without text. A pooled Gram stores the sum of source contributions. To subtract a deleted record, it needs that record's contribution, either stored separately or regenerated from its source. Our hybrid uses this second route where appropriate. Cold reconstruction extracts the retained features again. These are useful comparators because they

can share the same numerical kernels. The difference is the evidence available to those kernels, not permission to use a weaker reconstruction algorithm.

3 The reconstruction target

We now define the model that repair must recover. The specification fixes a pretrained checkpoint W, a set of identified token records $\mathcal{D}_0$, a teacher $T(W)$, and the tokenizer and record boundaries. It also fixes stage and coordinate order, rounding grids, positive ridge parameters, and the original normalization N_0. Deleting records changes membership, not these choices. The experiments use the original checkpoint as teacher, $T(W) = W$.

3.1 Source-local features

A stage is one calibrated affine transformation. For stage ℓ and record j, let

$$X_{\ell j} = F_\ell(T(W), j) \in \mathbb{Q}^{d_\ell \times t_j},$$

where d_ℓ is the feature width and t_j is the record's token count. F_ℓ is a declared finite CPU evaluation. Each successful binary64 output is interpreted exactly as a dyadic rational, an integer times a power of two. This specifies the arithmetic object being reconstructed without replacing the feature extractor by an ideal real-arithmetic network. Unsupported finite operations cause refusal.

The evaluator processes each record independently. For retained membership $R \subseteq \mathcal{D}_0$, concatenate its features in canonical source order to form $X_\ell(R)$. Its regularized second-moment matrix is

$$H_\ell(R) = \lambda_\ell I + \frac{X_\ell(R) X_\ell(R)^\top}{N_0}, \qquad \lambda_\ell > 0. \tag{1}$$

Here I is the identity matrix. The positive ridge makes the matrix positive definite. Keeping N_0 fixed ensures that subtracting records and reconstructing from retained records define the same target.

3.2 Adaptive weight decisions

Suppress stage and output-row indices. Factor the rational matrix as $H = L^\top \operatorname{diag}(\tau) L$, where L is unit lower triangular and the pivots τ_i are positive. This reverse LDL factorization determines the coefficient $c_{ih} = L_{ih}$ carrying an earlier rounding error into coordinate i. For a base-weight row w, define

$$v_i = w_i + \sum_{h<i} c_{ih}(w_h - q_h), \tag{2}$$

$$q_i = \operatorname{nearest}_{G_i}(v_i). \tag{3}$$

The adjusted value v_i is rounded to a code in the fixed grid G_i. An exact midpoint selects the lower code; endpoint codes saturate beyond the grid. The collection of all stage outputs is $Q_{\mathrm{fix}}(R)$.

For each experimental output row, the grid is $s_{\ell,r}\{-8, \ldots, 7\}$. Its scale is the smallest checkpoint-only dyadic scale with at most 24 significant binary digits that covers the base weights and gives finite exactly representable codes and midpoints. The scale is shared across that row's coordinates. Archive precision and weight precision are separate: the archive stores feature evidence, while these sixteen values define the deployed four-bit weights.

Earlier decisions enter later values through (2). A certificate must therefore establish the accepted prefix before using it to certify the next code. Across stages, the fixed teacher prevents repaired weights from changing the features that define later targets. Changing the teacher, original normalization, grids, evaluator, or ordering creates a different reconstruction problem.

4 Certifying decisions from lossy evidence

Preparation evaluates each source through the frozen teacher and encodes a canonical descriptor $D_{\ell j}$. Decoding returns entrywise bounds satisfying

$$\underline{X}_{\ell j} \le X_{\ell j} \le \overline{X}_{\ell j}. \tag{4}$$

Preparation checks every actual feature value against its bounds. Metadata binds the source and token identities, stage, target, codec, and a commitment to the original feature words.

The codec groups 256 token-major values into blocks. Each block stores a shared power-of-two step and signed 48-bit cell indices. Two-bit tags distinguish closed cells, exact grid values, and exact binary64 escapes near overflow. Endpoints are exactly representable dyadics. Escapes and metadata count toward storage, so 48-bit indices do not imply a uniform 25% archive saving.

4.1 From coefficient intervals to codes

A request first validates its predecessor, deletion IDs, retained membership, and target bindings. It

keeps each surviving descriptor unchanged and processes stages in order. A numerical solver proposes coefficients. A separate directed verifier bounds their errors over every feature realization in the decoded box. Using the accepted prefix in (2), it obtains an enclosure $[\underline{v}_i, \overline{v}_i]$ for the next adjusted value.

For a grid $g_0 < \cdots < g_{k-1}$, let $m_a = (g_a + g_{a+1})/2$. The decision cell for an interior code g_a is $(m_{a-1}, m_a]$. Endpoint cells extend to infinity. Acceptance requires the entire value enclosure to lie in one cell. The strict lower boundary and inclusive upper boundary implement the lower-code tie rule. A proposal close to a grid value is insufficient if its verified enclosure crosses a cell boundary.

If a box cannot certify the stage within its admitted work, the service may regenerate exact retained features and use a point certificate. It verifies that replayed words and containment match preparation. Access accounting includes every traversed ancestor stage. Temporary exact features do not replace the compressed successor descriptors. Exhausting a CPU, memory, refinement, or replay allowance ends the attempt without committing a complete replacement.

4.2 An executable coefficient bound

We give the link between a numerical solve and the adaptive coefficients. For one realized feature matrix X, set $K = XX^\top$, $\beta = \lambda N_0 > 0$, and $A = \beta I + K = N_0 H$. Let $S_i = \{i, \dots, d\}$ be the suffix beginning at coordinate i, and let e_1 be its first coordinate vector. Then

$$c_{ih} = K_{h,S_i} A_{S_i,S_i}^{-1} e_1, \qquad h < i. \tag{5}$$

To prove the identity, factor $A = L^\top D L$. Scaling H changes the diagonal factor, not L. With $S = S_i$, the suffix and cross blocks are

$$A_{S,S} = L_{S,S}^\top D_{S,S} L_{S,S},$$
$$A_{h,S} = L_{S,h}^\top D_{S,S} L_{S,S}.$$

Their product with $A_{S,S}^{-1} e_1$ is $L_{S,h}^\top L_{S,S}^{-\top} e_1 = L_{ih}$: the first column of the unit upper triangular inverse is e_1. Since $h < i$, $A_{h,S} = K_{h,S}$, proving (5).

Let $\widehat{z}_i$ be any numerical proposal. Suppose the verifier establishes $\|e_1 - A_{S,S}\widehat{z}_i\|_2 \le \eta_i$, where $\|\cdot\|_2$ is the Euclidean norm. The coefficient error obeys

$$|c_{ih} - K_{h,S}\widehat{z}_i| \le \eta_i \min\left\{ \frac{\|K_{h,S}\|_2}{\beta}, \frac{\sqrt{K_{hh}}}{2\sqrt{\beta}} \right\}. \tag{6}$$

For the first bound, write the error as $K_{h,S} A_{S,S}^{-1}(e_1 - A_{S,S}\widehat{z}_i)$ and use $A_{S,S} \succeq \beta I$. For the second, use $K_{h,S} = X_h X_S^\top$. Each singular value s of X_S contributes $s/(\beta + s^2)$ to the norm of $X_S^\top(\beta I + X_S X_S^\top)^{-1}$. This is at most $1/(2\sqrt{\beta})$, while $\|X_h\|_2 = \sqrt{K_{hh}}$. Multiplication by the residual bound proves the claim.

For uncertain features, directed arithmetic encloses the dot product $K_{h,S}\widehat{z}_i$, the residual norm, the Gram-row norm, and its diagonal uniformly over the feature box. Combining this dot-product enclosure with (6) bounds the coefficient. The argument applies to each realized feature Gram, which is positive semidefinite. It does not assign that property to arbitrary matrices inside an entrywise Gram enclosure. The coefficient intervals then feed the decision-cell check above.

4.3 How much uncertainty is tolerable?

The decision check gives a useful criterion for archive precision. Suppose the coefficient intervals for the accepted prefix have centers $\bar{c}_{ih}$ and radii $\delta_{ih} \ge 0$. Define the adjusted center and propagated radius as

$$u_i = w_i + \sum_{h<i} \bar{c}_{ih}(w_h - q_h), \tag{7}$$

$$\rho_i = \sum_{h<i} \delta_{ih} |w_h - q_h|. \tag{8}$$

For every admitted coefficient realization, $|v_i - u_i| \le \rho_i$ by the triangle inequality. If a proposed code has cell $(a_i, b_i]$, the sufficient acceptance conditions are

$$u_i - \rho_i > a_i, \qquad u_i + \rho_i \le b_i. \tag{9}$$

At endpoint codes the corresponding infinite boundary imposes no restriction. These expressions are exact arithmetic identities; an executable evaluation must enclose the centers, products, sums, and comparisons outwardly as well.

This criterion separates two sources of difficulty. Feature uncertainty widens coefficient intervals. Earlier weight residuals $w_h - q_h$ determine how strongly that width affects the current decision. A small archive error can matter when the adjusted value lies near a midpoint, while a larger error can be harmless far from it. There is therefore no single feature-error threshold that certifies every coordinate independently of the accepted prefix.

The triangle-inequality bound also explains possible conservatism. Coefficients share the same

underlying features, but a product of coefficient intervals can include combinations that no feature realization attains. The resulting value interval can cross a boundary even when the exact target is far from it. A refusal consequently identifies insufficient evidence under the current bounded certificate; it does not imply that the target code is undefined or incorrect. Point replay removes feature uncertainty while retaining the requirement to verify numerical error. It can resolve this conservatism without changing either the target or the output contract.

4.4 Choosing the numerical representation

The same coefficient has the token-space form

$$c_{ih} = X_h(\beta I + X_S^\top X_S)^{-1} X_i^\top. \qquad (10)$$

This follows by multiplying $X_S^\top(\beta I + X_S X_S^\top) = (\beta I + X_S^\top X_S)X_S^\top$ by the respective inverses. The two identities allow a solver to work in feature or token space without changing the target.

For width d, retained tokens T, and output rows r, token-space coefficient construction has a leading structural term proportional to dT^2. Feature-space construction has terms proportional to $d^2T + d^3$, followed by row work proportional to rd^2. The service admits its arrays and bounded schedule before feature access. Its original dispatcher prefers token space when $4T \le d$, and streamed feature space otherwise. The final matched study fixes the token route for all point-valued solves. These are declared engineering policies. Actual runtime also depends on interval width, decoding, repeated verification, and replay. Compatible controls receive the same kernels and route choices.

5 Correct models and reusable successor states

The first guarantee concerns the model produced by a successful request.

Theorem 1 (Exact code recovery)**.** *Assume that features come from the frozen source-local evaluator and that trusted descriptors satisfy* (4)*. Each accepted certificate must be valid for every feature realization in its supplied box, conditional on its accepted prefix. Any fallback must reproduce the same finite features and certify* (2)*–*(3)*. If the service commits every required stage, the resulting model is* $Q_{\mathrm{fix}}(R)$*.*

Proof. Source locality makes each surviving feature matrix equal to its retained-only reconstruction. It lies inside the prepared box. A valid first-coordinate certificate therefore selects the target code. Suppose the earlier accepted codes equal the target prefix. The next certificate's premise is then satisfied, so its selected code also equals the target. Induction establishes each row. A successful fallback uses the same features and target, giving the same conclusion. The fixed teacher makes this argument apply to every stage without changing later defining inputs. The complete extent check covers the entire committed model. □

The state must also support the next request. Fix the codec, preparer, representation rule, source ordering, provenance policy, and serialization. Let $E(R)$ contain the retained canonical descriptors or the canonical exact pooled Grams required by that representation. Define

$$\mathcal{S}(R) = \mathrm{encode}\big(\text{target}, Q_{\mathrm{fix}}(R), R, E(R), \text{provenance}\big). \qquad (11)$$

Provenance here means retained-record identity and target/codec bindings. Execution logs are external. Required trust sidecars and source-retrieval obligations are counted in the service's storage and access contract.

Lemma 1 (Exact source subtraction)**.** *For fixed source-local features,* $K(R) = \sum_{j\in R} X_j X_j^\top$*. Exact subtraction of trusted deleted-source contributions, with original normalization unchanged, equals fresh retained-source accumulation.*

Proof. Each source contribution is unchanged by other records' deletion. Finite sums over disjoint source sets are additive in rational arithmetic. Subtracting the removed summands leaves the retained sum. Canonical integer scaling and serialization remove dependence on the order of additions and deletions. □

Theorem 2 (Canonical successor equality)**.** *Under Theorem 1, suppose descriptors are deterministic and retained unchanged, Gram updates satisfy Lemma 1, and compared executions use the same policies in* (11)*. A successful repair returns* $\mathcal{S}(R)$*. Its bytes equal those of any successful certified fresh reconstruction in that representation. The equality holds after every step of a sequence of successful valid deletions.*

Proof. The model agrees by Theorem 1. Retained descriptors agree because their source-local inputs and encoder agree. Pooled Grams agree by exact subtraction and canonicalization. Membership, target, provenance, and encoding policies are identical, so each serialized component agrees. Apply the same argument at each successor. □

The equality is representation-specific: a Gram archive and an interval archive can encode the same model with different state bytes. It also leaves execution histories distinct. The trusted computing base consists of the finite evaluator, directed arithmetic, compiler/runtime assumptions, source provenance, and immutable inputs during verification. Hash commitments detect changed bytes; they do not prove containment or authenticate a hostile execution host. Positive ridge guarantees mathematical invertibility, while the bounded certificate may still refuse a particular request.

6 Evaluation design

We evaluate three questions separately: whether a complete repair is exact, what resources it consumes, and how the fixed-teacher target affects predictions. The service studies use DistilGPT2, with approximately 82 million parameters, and GPT2, with 124 million (Hugging Face, n.d.-a; Hugging Face, n.d.-b). They calibrate 24 and 48 affine stages, respectively. The packed outputs contain 42,467,328 and 84,934,656 codes; unchanged checkpoint components remain necessary for inference. Calibration records are English C4 web text (Raffel et al., 2020), each with 128 tokens.

6.1 Comparators and complete accounting

The *cached lossless* arm stores exact feature values. The *modern lossless* arm selects the smallest complete descriptor among raw, zlib6, byte-shuffled zlib6, Zstandard3, and byte-shuffled Zstandard3 encodings, with fixed-order ties. Every candidate encoder is charged. *Cold reconstruction* extracts retained features from source records. The *hybrid exact-Gram* arm stores exact Grams at eighteen width-768 stages and lossless features at six width-3072 stages in DistilGPT2. Our *compressed* arm stores the 48-bit interval descriptors. All arms target the same codes and are compared with a fresh retained-input reconstruction in their respective state representation.

An oracle traverses retained tokens without reading the candidate predecessor. The audits compare actual model and state payloads; the later studies also compare required trust artifacts. The oracle shares the numerical solver. This checks complete reconstruction and source independence, while leaving implementation errors shared by that solver outside the comparison.

A request begins with loading and validating the predecessor and ends after complete output and cleanup. Its cost includes attempted certificates, exact accumulation, source extraction, fallback, and serialization. In the optimization and matched studies, complete host time also includes admission and container startup. We retain disjoint timing partitions and do not add a nested source clock to the stage clock that contains it.

Preparation must be included when comparing deployment lifetimes. If P_a is preparation for representation a and $C_{a,k}$ is its complete cost on actual successor transition k, then

$$L_a(m) = P_a + \sum_{k=1}^{m} C_{a,k}. \tag{12}$$

Alternative deletions from an unchanged root do not form such a trajectory. Shared preparation has an explicit attribution convention rather than an implicit zero cost.

Live storage includes codes, descriptors or Grams, targets, tokens or retrieval indexes, provenance, and required trust sidecars. We also report a total that adds the common checkpoint. A model embedded in the state is counted once. Runtime and codec installations, and physical audit archives, are separate. Source access is measured as source–stage record traversals. Thus replaying one record through twenty-four stages costs twenty-four accesses, even if only the last stage needed its exact features.

For a stage of width d with T retained tokens, a feature archive represents dT values. A pooled symmetric Gram represents $d(d+1)/2$ entries before its exact-number encoding and metadata. These counts explain why an aggregate can be appealing at larger token counts yet less attractive after many deletions. They do not determine byte size: exact accumulators, interval tags, lossless compressibility, and per-source indexing have different overheads. Retaining a separate Gram contribution for every source would also change the hybrid's access and storage contract. We there-

fore compare complete serialized states rather than multiplying a nominal precision by these entry counts.

6.2 Four studies and their selection history

The first complete service study uses thirteen previously unexposed training records. It deletes one and then six more, giving $13 \rightarrow 12 \rightarrow 6$ records while keeping normalization 1664 and ridge $1/100$. Records are sorted by ID; the first is deleted, followed by the next six. It compares all five arms. The second study holds that root and target fixed and evaluates reference and optimized versions of the compressed and modern lossless arms. Both optimized arms receive the compatible token-neighbor kernel. Compression adds a native decoder; modern lossless adds bounded reuse of decoded descriptors. Each package is evaluated as a whole.

The third study matches both checkpoints on four exposed records, deletion $4 \rightarrow 3 \rightarrow 2$, normalization 512, and ridge $1/100$. Every point-valued solve uses the existing token route, fixed before execution for all four cells and both fresh oracles. Box and Gram policies remain unchanged. Each request runs in a fresh process on the same host with one CPU and 16 GiB memory. The approved grid allowance is 1,327,104 entries. Sealed preparation artifacts are reused and their costs retained. All timing cells in these studies are single observations. Execution order is fixed, with endpoint reversal in the optimization screen; OS cache is uncontrolled.

The fourth study evaluates the earlier 1664-normalized DistilGPT2 target on untouched articles. It is distinct from the smaller matched targets. Teacher and precision choices preceded it: a nearest-rounded teacher had failed a pooled quality guard with a 5.92% perplexity increase; the original-checkpoint teacher was initially selected using eight exposed articles. A 40-bit archive refused certification, followed by a separately registered 48-bit study. The initial matched GPT2 request also refused after both preparations completed. A separate stage diagnostic motivated the prospective point-token policy above. These choices make the later studies scoped tests of fixed configurations.

7 Complete repair results

7.1 Storage and source access at a fixed target

All ten arm–endpoint comparisons in the disjoint-root service study match fresh reconstruction for the complete model and successor state. Table 1 shows their resource choices. Compression reduces live storage relative to both lossless caches and needs no source traversal on this trajectory. Relative to modern lossless, live savings are 9.24% and 8.70%; adding the checkpoint reduces these to 4.71% and 3.10%. The codec installation adds 23,870,575 bytes separately.

The hybrid demonstrates why the access contract matters. At twelve records, it is both smaller and faster than compression, while reading source records. At six, compression is smaller and faster than the hybrid and avoids its 144 accesses. Both lossless caches are faster than compression at both endpoints. The compressed requests use bounded numerical fallback on 68 and 2 rows, respectively, but no source replay. Numerical refinement and source access are therefore distinct costs.

7.2 Optimizing both sides of the comparison

All eight requests in Table 2 match fresh model, state, and trust artifacts, with no source replay. The compressed package reduces observed request times by 11.53% and 36.17%; the modern package reduces them by 0.69% and 19.93%. The first optimized endpoint is nearly tied. The second favors modern lossless. Their stored-state sizes are unchanged from Table 1.

Phase measurements explain the endpoint difference. At six records, compressed stage work takes 803.754 seconds versus 680.251 for modern lossless. Its cheaper predecessor validation, 5.319 versus 33.142 seconds, does not offset that gap. At twelve records, stage work is nearly tied at 3,884.539 versus 3,891.548 seconds, while compressed validation saves 30.030 seconds. Including attributed shared preparation and both requests, the totals are 10,995.918 seconds for compression and 10,287.226 for modern lossless. That observed lifetime favors the lossless archive. The bundled changes and phase clocks do not isolate a decoder-only effect.

Records	Service arm	Request (s)	Live (MB)	With checkpoint (MB)	Accesses
12	Cached lossless	4,419.007	368.003	720.828	0
	Modern lossless	4,579.496	367.456	720.282	0
	Cold reconstruction	5,973.008	368.003	720.828	288
	Hybrid exact Gram	4,921.935	318.918	671.743	24
	Compressed	5,206.569	333.497	686.322	0
6	Cached lossless	922.484	195.615	548.441	0
	Modern lossless	897.841	195.330	548.155	0
	Cold reconstruction	1,619.473	195.615	548.441	144
	Hybrid exact Gram	1,694.311	220.247	573.072	144
	Compressed	1,351.098	178.333	531.158	0

Table 1: Complete DistilGPT2 service on $13 \to 12 \to 6$ records. MB are decimal; live storage includes trust sidecars. Accesses count source–stage record traversals during the request. The checkpoint column excludes runtime and codec installation. Each row is one observation; every model/state comparison passes.

Records	Archive	Ref. (s)	Opt. (s)
12	Compressed	4,444.116	3,931.646
	Modern	3,995.241	3,967.523
6	Compressed	1,328.624	848.119
	Modern	939.553	752.336

Table 2: Complete host times for both optimization packages on the same DistilGPT2 trajectory. Modern denotes the modern lossless cache. Each timing cell is one request.

7.3 Matched repair across two checkpoint depths

All sixteen complete model, state, and trust comparisons pass for the matched DistilGPT2/GPT2 study. Both audits cover every calibrated stage and code, including the second successor. Table 3 reports all request times. The optimization improves each representation relative to its reference, but optimized compression takes longer than optimized modern lossless at all four checkpoint/endpoint settings.

Storage savings persist across both depths (Table 4). Source avoidance is less uniform. At three GPT2 records, both compressed cells replay 144 source–stage record pairs after refusal at the final-block MLP up-projection. Optimized compressed stage work takes 1,458.767 seconds, including 695.104 seconds of source extraction, versus 476.578 seconds for modern lossless. The other fourteen requests perform no source traversal or fallback.

Replay explains part of the largest latency gap, but not all slower requests. At two DistilGPT2 records, compressed stage work takes 267.560 seconds versus 114.651 for modern lossless despite zero replay. Validation is cheaper at 2.266 versus 9.120 seconds. Thus reducing archive validation work does not necessarily pay for certifying uncertain features. These observations motivate a storage-led use case rather than a general acceleration claim. The tested models belong to the same GPT family, and single timing observations do not estimate latency variability across deployments.

7.4 Predictive cost of the fixed teacher

Quality is evaluated for the earlier DistilGPT2 target with normalization 1664. We select 320 articles without observing losses from 2,778 eligible records in a saved frame of 3,801 complete records. The entire inspected frame, sixty historical exclusions, and the separate calibration frame are protected against reuse. Each article supplies 127 predictions from 128 tokens. Six model roles share these articles: the fixed-teacher and sequential models at each retained endpoint, the original checkpoint, and a nearest-grid control.

Sequential calibration uses the same retained records, original normalization, and grids. A fixed-seed paired article bootstrap uses 20,000 draws, shared across endpoints. The frozen guards require each pooled perplexity ratio and its one-sided 97.5% upper bound to be at most 1.05, and each observed maximum article ratio to be at most 1.20. The two upper bounds have nominal family-wise level 95%. This is an approximate bootstrap over the admitted frame; the observed maximum describes those articles.

The ratios in Table 5 show a small predictive cost. Candidate perplexities are 60.1053 and 60.3088, compared with sequential values

Checkpoint	Records	Ref. comp.	Opt. comp.	Ref. lossless	Opt. lossless	Opt. C/L
DistilGPT2	3	589.239	411.279	369.644	286.419	1.4359
	2	436.691	325.131	219.802	179.706	1.8092
GPT2	3	2,027.290	1,544.049	733.046	580.147	2.6615
	2	838.153	615.039	406.506	326.341	1.8847

Table 3: Complete host times (seconds) for the matched $4 \to 3 \to 2$ point-token study. Lossless denotes the modern cache. C/L divides optimized compressed time by optimized lossless time. Each cell is one observation, with complete equality against its same-checkpoint fresh reconstruction.

	Live (%)		Total (%)	
Records	3	2	3	2
DistilGPT2	7.78	7.01	1.84	1.30
GPT2	7.86	7.09	2.24	1.61

Table 4: Matched compressed storage savings relative to modern lossless. Total adds the shared checkpoint to each live-state size; it excludes runtime installs.

Records	Pooled ratio	Upper bound	Maximum
12	1.005421	1.010342	1.129140
6	1.007149	1.011431	1.126961

Table 5: Untouched quality: fixed-teacher perplexity divided by matched sequential perplexity on 320 articles. The upper bound is one-sided 97.5%. All prespecified guards pass.

59.7812 and 59.8807. The paired 95% percentile intervals for the ratios are [1.000455, 1.010342] and [1.002870, 1.011431], both above one. Original-checkpoint and nearest-grid control perplexities are 54.9643 and 74.6245. These measurements support the specified tolerance for this target and article frame. The 512-normalized targets in the matched study have no quality evaluation.

All twenty-four CPU/CUDA reference comparisons pass the fixed 10^{-8} mean negative-log-likelihood barrier, with maximum difference 1.155×10^{-14}. The terminal audit checks 1,920 GPU and twenty-four CPU losses, saved sequential codes, original parameters, and recomputed statistics. This evaluates identical saved models across likelihood implementations; calibration itself uses the finite CPU evaluator defined in Section 3.

8 Relation to prior work

Calibration and retained information. GPTQ (Frantar et al., 2023) motivates adaptive low-bit rounding guided by calibration second moments. Our target freezes the feature teacher so that deleting one record leaves every other source contribution defined in the same way. Quantized sufficient statistics provide another route to exact deletion: Tavory et al. (2026) study frozen schemas and additive prediction corrections. Our certificates instead resolve an adaptive sequence of weight decisions from uncertain features. Exact additive subtraction remains useful here as a control and as one state representation.

Quantization and exact deletion. Ginart et al. (2019) use quantized k-means, retained metadata, and stability tests to update a model when deletion leaves quantized centroids unchanged, with retraining otherwise. Exact-Fun (Xiong et al., 2023) corrects deleted gradients in a quantized federated procedure and resumes training from an affected round. These works establish that quantization can aid exact deletion and that fallback is a useful part of the design. The distinguishing requirement here is universal recovery of adaptive codes, including changed ones, when the retained calibration features themselves are only known through enclosures.

Compression and verification. MEDU (Lang et al., 2026) stores compressed distributed-training updates and bounds deviation from retraining. Our output contract is equality of discrete codes. QEBVerif (Zhang et al., 2023) bounds inference-output differences between a network and its quantized counterpart using reachability analysis and MILP. Here the verified property belongs to the calibration procedure that generates weight codes. ExecCert (Zhao et al., 2026) separates candidate generation, release contracts, and executable verification, including residual certificates and finite output formats. We instantiate that broader separation with a uniform feature-box certificate, an adap-

tive rounding target, and a successor-state contract. The contribution is this combination and its measured operating choices, rather than a new general interval-verification principle.

9 Reproducibility and practical use

Implementation and data. The CPU runtime uses CPython 3.12.14, NumPy 2.3.5, OpenBLAS 0.3.30, gmpy2 2.3.2, tokenizers 0.23.2, GCC 12.2, and Zstandard 1.5.7 through python-zstandard 0.25.0 on x86-64 Linux. Quality inference uses an NVIDIA A100 80GB; bootstrap statistics use NumPy 2.2.6. Model citations pin checkpoint revisions. The C4 revision is `1588ec454efa`. The thirteen-record root comes from complete records in the first 2 MiB compressed prefix of English training shard 2. Quality uses an 8 MiB decompressed frame of validation shard 1. Records use the first 128 tokens without beginning/end markers, with positions reset per source. Selection excludes exact IDs, text/normalized-text/URL hashes, and token prefixes of lengths 16, 32, 64, and 128. Sampling is within these bounded frames, not uniform across C4. Earlier development included WikiText-2 (Merity et al., 2017).

Cost of the research runs. The service study uses 47,767 numerical CPU seconds and 51,356.452 scheduler CPU seconds over fifteen allocations. Optimization uses 35,258 numerical seconds and [39,389.110, 39,396.116) scheduler seconds. The matched continuation uses 14,222 new numerical seconds; its preserved failed-screen and diagnostic debit of 6,304 makes 20,526. Its twenty-one allocations use [18,213.235, 18,214.255) scheduler seconds. Shared preparation took 1,389.528 host seconds for DistilGPT2 and 2,992.098 for GPT2. Quality workers account for 7,526 CPU seconds and 7,588.049 scheduler seconds including accounting recovery. Worker and scheduler clocks overlap. A historical unresolved 12,602-second reservation remains separate, so these figures are not a complete single all-history total.

The archive retains refused certificates, a construction timeout, quota and audit failures, the exclusion-audit admission failure, and accounting recovery. The failed matched attempt completed no request and cancelled nineteen unstarted dependents. Its follow-up certified one stage's 2,359,296 codes without a fresh stage oracle; it is recorded as a diagnostic.

Inputs and responsibility. A versioned source and evidence archive has been prepared. It preserves actual output comparisons, scalar losses, frozen selections, and 1,322 source/protocol copies. It is an author-side reproducibility record, not an external review supplement. Source release terms remain to be assigned; release would exclude raw corpus text, token bodies, and model/runtime binaries. Fresh reproduction requires the pinned external inputs and bound runtime, and has not been executed from the portable archive. Model cards declare Apache 2.0 terms for DistilGPT2 and MIT for GPT2. C4 declares ODC-BY plus Common Crawl conditions; these do not establish rights to every underlying web page.

Logical deletion from the live calibration state leaves external corpora and audit archives intact. Public web text can contain personal or offensive content; we did not perform a content-level privacy or demographic audit. Generative AI assisted planning, software development, analysis, literature checks, and writing. The authors remain responsible for the methods, results, references, and text.

Choosing an archive. For a deployment that prioritizes latency and can retain exact features, the optimized lossless cache is the strongest measured choice. If live feature storage is the binding cost, certified compression offers a smaller state while preserving the exact reconstruction contract. Its service policy must specify what happens when source replay is needed but unavailable. If source access is acceptable, the hybrid Gram representation is a competing option. This choice should use total deployed bytes and preparation-inclusive request costs, not payload bit width alone.

10 Conclusion

The information needed to reconstruct a low-bit model can be smaller than the information needed to reconstruct its calibration features. A feature enclosure is sufficient whenever it resolves every adaptive rounding decision. Coupling that test with exact replay and canonical state construction gives a repair service whose successful outputs remain usable for later deletions. The empirical comparison identifies the practical price of that guarantee: modest storage savings, workload-dependent source access, and extra certification work.

References

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. ICLR. Primary preprint: arXiv:2210.17323, version 2.

Antonio A. Ginart, Melody Y. Guan, Gregory Valiant, and James Zou. 2019. *Making AI Forget You: Data Deletion in Machine Learning*. Advances in Neural Information Processing Systems 32. Primary proceedings paper.

Hugging Face. n.d.-a. DistilGPT2 model card. Model repository, pinned revision `2290a62682d0`.

Hugging Face. n.d.-b. GPT-2 model card. Model repository, pinned revision `607a30d783df`.

Natalie Lang, Alon Helvitz, and Nir Shlezinger. 2026. Memory-efficient distributed unlearning. *IEEE Access*, 14:24361–24378.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Journal of Machine Learning Research, 21(140):1–67. Primary journal article.

Ami Tavory, Shripad Gade, Tal Sarig, Noam Touitou, and Ido Guy. 2026. *Exact Unlearning via Quantized Sufficient Statistics*. arXiv:2610.07197, version 1.

Zuobin Xiong, Wei Li, Yingshu Li, and Zhipeng Cai. 2023. Exact-Fun: An exact and efficient federated unlearning approach. In *IEEE International Conference on Data Mining*, pages 1439–1444.

Yedi Zhang, Fu Song, and Jun Sun. 2023. QEBVerif: Quantization error bound verification of neural networks. In *Computer Aided Verification*, volume 13965 of LNCS, pages 413–437. Springer.

Ziyu Zhao, Xinyu Wang, Xiao-Wen Chang, and Yixuan He. 2026. *From Mathematical to Executable Certificates for Machine Unlearning*. arXiv:2610.02268, version 1.